Lecture 5

Tuesday February 3rd 2026

Processes

- Representation
- Creation
- Termination
- Switching
- Queues
- Scheduling
- Inter-process communication

Process Concept

- A process is a program in execution
- A process image consists of three components
  - An executable program
  - The associated data needed by the program
  - The execution context of the process

Process Control Block (PCB)

- **PCB** is included in the context along with the stack
- It is a "snapshot" that contains all necessary and sufficient data to restart a process where it left off
- On entry in the operating system's **process table** (array or linked list)
- Can view the Process Block if you find a process id with top then:
  - `cat /proc/<pid>/status`

Stack

- LIFO Last in First Out
- Data is loaded into the stack from the bottom up
- When undoing an action you are just popping it from the stack
- Useful when doing multi level functions and recursion
- Stack is used to organize the function calls
  - Issues like creating an infinite loop will cause a stack overflow error

Process State

Process will move through different queues (ready queue, waiting queue)

- **New** - process is being created
- **Ready** - process is waiting to be assigned to a processor
- **Running** - instructions are being executed
- **Waiting** - the process is waiting for some event to occur (e.x. user input)
- **Terminated** - The process has finished execution

Process Creation

- Some events that lead to process creation (enter)
  - The system boots:
    - when system is initialized, several background processes of "daemons" are started (email, logon, etc)
  - User requests to run an application
  - An existing process spawns a child process
    - server process may create a new process for each request handled
- child processes are created by spawning

Watch recorded lecture for example of child spawning code

- `int rc = fork()` - will create an identical child process with a different pid
- the original process is called the parent
- If parent the fork() will return a value greater than zero
- The child will have a value equal to zero so this is how you are able to have the branch logic (based on the value of the rc variable in the separate programs)
- The child process will start execution at the point after the fork in the parent program